

Introduction to R

Session 06: Data Relations

Álvaro Pérez

Instituto Tecnológico Autónomo de México

Spring 2025

Overview of Today's Session

Mutating Joins

Filtering Joins

Binding Joins

Set Operations

Mutating Joins

Mutating joins act like the `mutate()` function in that they add new columns to one table based on values in another table.

All the mutating joins have this basic syntax:

```
join(x, y, by = NULL, suffix = c(".x", ".y"))
```

- ▶ **x** = the first (left) table
- ▶ **y** = the second (right) table
- ▶ **by** = what columns to match on. If you leave this blank, it will match on all columns with the same names in the two tables.
- ▶ **suffix** = if columns have the same name in the two tables, but you aren't joining by them, they get a suffix to make them unambiguous. This defaults to `".x"` and `".y"` but you can change it to something more meaningful.

Mutating Joins

`left_join()`

A **left_join** keeps all the data from the first (left) table and joins anything that matches from the second (right) table. If the right table has more than one match for a row in the right table, there will be more than one row in the joined table.

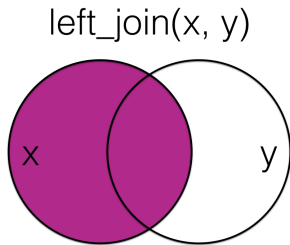


Figure 1. Left Join

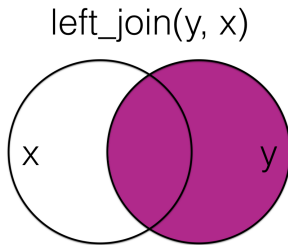


Figure 2. Left Join

Mutating Joins

`right_join()`

A **right_join** keeps all the data from the second (right) table and joins anything that matches from the first (left) table.

```
1 right_join(subject, exp, by = "id")
```

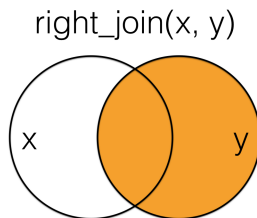


Figure: Right Join

Mutating Joins

`inner_join()`

An **inner_join** returns all the rows that have a match in the other table.

```
1 inner_join(subject, exp, by = "id")
```

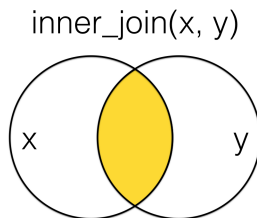


Figure: Inner Join

Mutating Joins

full_join ()

A **full_join** lets you join up rows in two tables while keeping all of the information from both tables. If a row doesn't have a match in the other table, the other table's column values are set to NA.

```
1 full_join(subject, exp, by = "id")
```

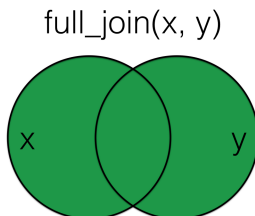


Figure: Full Join

Filtering Joins

Filtering joins act like the `filter()` function in that they remove rows from the data in one table based on the values in another table. The result of a filtering join will only contain rows from the left table and have the same number or fewer rows than the left table.

Filtering Joins

`semi_join()`

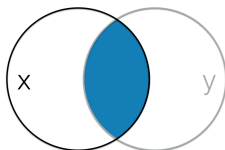
A **semi_join** returns all rows from the left table where there are matching values in the right table, keeping just columns from the left table.

```
1 semi_join(subject, exp, by = "id")
```

Order matters in a semi join.

```
1 semi_join(exp, subject, by = "id")
```

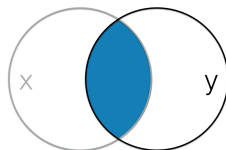
`semi_join(x, y)`



*only columns from x; no duplicate rows, even if >1 match in y

Semi Join

`semi_join(y, x)`



*only columns from y; no duplicate rows, even if >1 match in x

Semi Join

Filtering Joins

anti_join()

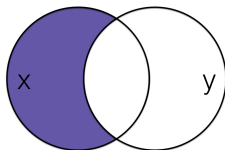
An **anti_join** returns all rows from the left table where there are not matching values in the right table, keeping just columns from the left table.

```
1 anti_join(subject, exp, by = "id")
```

Order matters in an anti join.

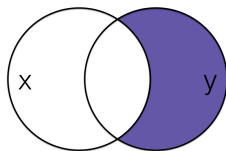
```
1 anti_join(exp, subject, by = "id")
```

anti_join(x, y)



Anti Join

anti_join(y, x)



Anti Join (Reversed)

Binding Joins

Binding joins bind one table to another by adding their rows or columns together.

1. **bind_rows()**: You can combine the rows of two tables.
2. **bind_cols()**: You can merge two tables with the same number of rows. This is only useful if the two tables have their rows in the exact same order. The only advantage over a left join is when the tables don't have any IDs to join by and you have to rely solely on their order.

```
1 new_info <- tibble(  
2   colour = c("red", "orange", "yellow", "green", "  
3             blue")  
4 )  
bind_cols(subject, new_info)
```

Set Operations

Set operations compare two tables and return rows that match (intersect), are in either table (union), or are in one table but not the other (setdiff).

- ▶ **intersect()**: returns all rows in two tables that match exactly. The columns don't have to be in the same order.
- ▶ **union ()**: returns all the rows from both tables, removing duplicate rows.
- ▶ **setdiff ()**: returns rows that are in the first table, but not in the second table.